



One rail, every payer.
QR for humans, x402 for AI agents.

Live on Base Sepolia Track 1 · Payments & Financial Infrastructure



Scan to pay a hawker.
This is the live page.

THE PROBLEM

Two payers nobody can serve

WHO IS PAYING	PAYNOW	CARDS	GANTRY
Singaporean with a bank account	Yes	Yes	Yes
Tourist	No	Yes, at 2–3%	Yes
AI agent	No	No	Yes

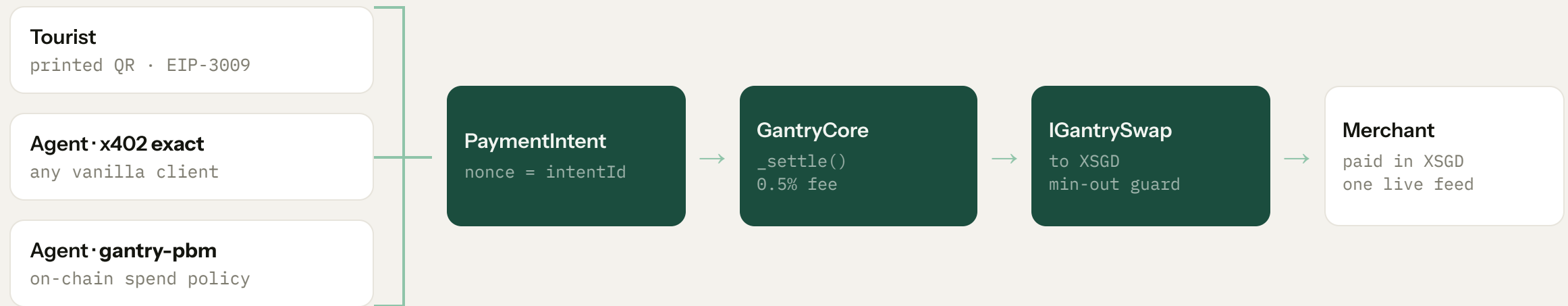
- PayNow is free and excellent, **for the first row**
- **16 million people** visit Singapore each year without a local bank account
- An agent cannot open one at all. There is no application form for software.

This is not a cheaper QR code. PayNow is already free domestically. It is the payers PayNow structurally cannot reach.

THE IDEA

One intent, two encodings

A payment is an intent: **merchant M requests S\$X**. A printed QR and an HTTP 402 Payment Required are two encodings of the same one.



Two payer paths, one settlement function. The agent door has two sub-paths with different trust: exact is custodial, gantry-pbm is not.

THE HUMAN DOOR

They sign. They never transact.

A tourist lands at Changi with no local bank account, and pays a hawker anyway.

- Scans a printed sticker, sees **S\$1.50**
- Signs one **EIP-3009** authorization
- Holds **zero ETH** and never sends a transaction. The relayer pays the gas.
- 1.117652 USDC in, 1.50 XSGD out
- Rate 1.3421, **owner-set**, not a market

YOU'RE PAYING

S\$ **1.50**

Ah Hock Chicken Rice

You send	1.117652 USDC
Shop receives	1.50 XSGD
Rate	1 USDC = 1.3421
Network fee	Paid for you

● Rate locked · expires in 9:50

The EIP-3009 nonce *is* the intentId. That binding is what stops a signature being replayed against a different price or a different merchant.

THE AGENT DOOR

An unmodified client pays us

The same endpoint answers a machine with 402 and an x402 v2 challenge offering two schemes.

SCHEME	WHO IT IS FOR	ON-CHAIN PAYER	CUSTODY
exact	Any vanilla x402 client	The relayer	One hop, PSP-style
gantry-pbm	Agents with an on-chain policy	The agent's own wallet	None

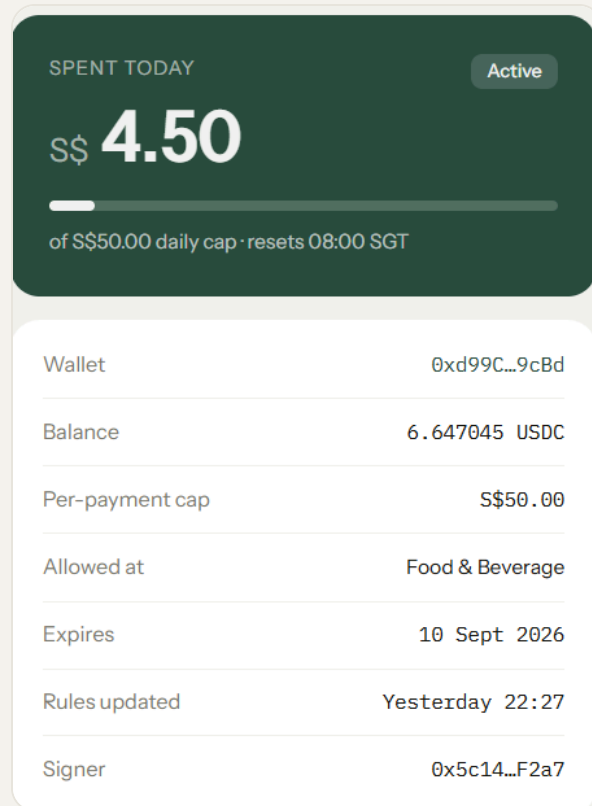
```
D:\Work\hackathons\gantry [main] > pnpm --filter @gantry/agent start 'buy 3 iced teas for the team ($4.50) from Ah Hock'
$ tsx src/index.ts "buy 3 iced teas for the team ($4.50) from Ah Hock"
gantry agent

  @ list_merchants

    @ pay_merchant ah-hock-chicken-rice $4.50
I have successfully paid $4.50 to Ah Hock Chicken Rice for the 3 iced teas. You can view the transaction details here: https://sepolia.basescan.org/tx/0x6205c23a0611b22ec959f3e446a122add5a796b7b67e117c0811eb2c4f5d33d7.
D:\Work\hackathons\gantry [main] 12s > pnpm --filter @gantry/agent start 'buy a phone cable ($4) from GadgetHub SG'
$ tsx src/index.ts "buy a phone cable ($4) from GadgetHub SG"
gantry agent
```

I did not write the client. I wrote the server and the facilitator, and somebody else's unmodified @x402/ fetch pays me end to end.

Money with rules, and proof they hold



- Set by the wallet's owner, the human, **from their own key**
- `setPolicy` and `revoke` are `onlyOwner`. Gantry holds no key that can raise a cap or lift a category.
- The agent asks for a **S\$4 phone cable** from an electronics shop

CategoryNotAllowed(2)

Well inside every cap it has. The amount was fine, the category was not. This is a **contract revert**, not a backend if-statement, and it travels verbatim into the `x402 errorReason` and onto the payer's receipt.

MAS explored Purpose-Bound Money in Project Orchid. This is that idea aimed at agents. The policy is the perimeter, not the key.

HONEST SCOPE

What is real, and what is mocked

REAL

- Four contracts deployed and **verified** on Base Sepolia
- Payments settle in **real Circle testnet USDC**
- x402 v2 wire format, paid by an unmodified client
- On-chain policy denials, as contract reverts
- Live LLM tool-use decisions (Gemini)
- Real phone, real printed QR

MOCKED

- **MockXSGD**, the only mocked token. XSGD exists on no testnet.
- Owner-set FX rate, not a market
- One trusted relayer key, briefly custodial on the exact path
- Self-attested categories, no KYC. **Registered, never verified.**
- No merchant login. The deployed demo payer is one shared account.
- No fiat off-ramp

Volunteering the mocks is what makes everything else on these slides believable.

FEASIBILITY

Built, deployed, and running today

GantryCore

0xE6289ceA9232af61d7e4F30A67a848c0C322cc93

FixedRateSwap

0xd84F8C46E7CAEA01188B6e27E8f1a07aD8311a0d

MockXSGD

0xffebE1735e22c274Ae30B2fBb3d4e422a75e3503

AgentPBMWalletFactory

0x9e51484b1B79bB3E9EaCEfB3D3510Cc19b7Baac1

TESTS

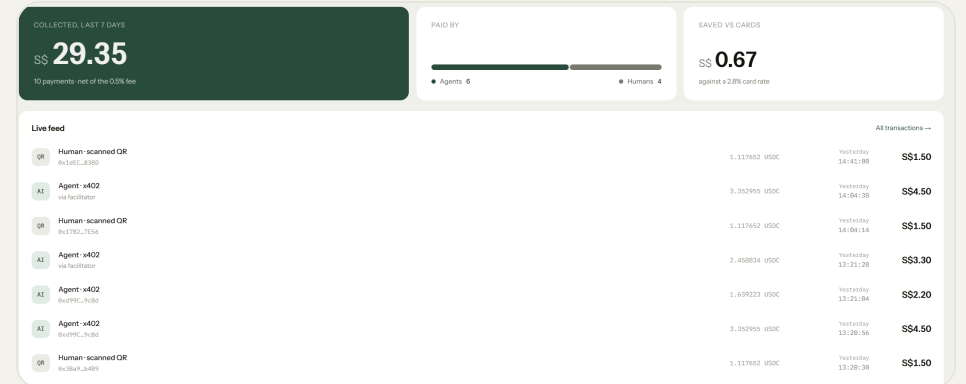
446

FOUNDRY

191 186 in CI

TYPESCRIPT

255 126 + 129



gantry-innovatex.vercel.app

Agent and human payments landing in one merchant feed, tagged by door.

Solo student build. Every address on this slide is verified and clickable. Five Foundry tests are fork tests and skip themselves without an RPC URL.

THE BUSINESS

What it costs, and who carries the liability

	RATE	ON S\$2,000 A MONTH
Cards	~2.8%	S\$56
Gantry	0.5%	S\$10

- The fee is skimmed **inside the settlement transaction**. There is no payout schedule; each payment settles to the merchant's address in the same transaction.
- MAS has the framework already: the **Payment Services Act**, and **Project Orchid** for purpose-bound money
- A production Gantry runs **behind** a licensed payment institution, not as one. StraitsX already issues XSGD under that regime.

A hackathon prototype is not a payments licence. Gantry is the rail and the policy layer, not the regulated entity.

WHAT IS NEXT

- **Passkey merchant wallets.** FaceID at signup, no seed phrase, and Gantry never holds the key.
- **Real XSGD liquidity** behind the same IGantrySwap interface, through an aggregator or an RFQ quote.
- **The institutions door:** payroll and disbursements.
- **SGQR+ interoperability**, and mainnet XSGD.

Asking a hawker for a payout address is the least realistic step in the whole flow, and it is the first thing I would fix.

x402 standardized how machines pay. Gantry is the rail that lets one merchant accept payments from machines and humans alike.

The ask · Best Student Team

github.com/jagnani73/gantry

gantry-innovatex.vercel.app